

0.8.2. Tout l'alphabet, avec deux boucles — doublé, deux positions

Tout l'alphabet deux fois : lignes 0 et 1, puis lignes 5 et 6.

1. Ce qu'il faut comprendre

C'est le 0.8 et le 0.8.1 réunis : l'alphabet du 0.8 (lignes 0 et 1), puis celui du 0.8.1 (lignes 5 et 6).

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1526 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

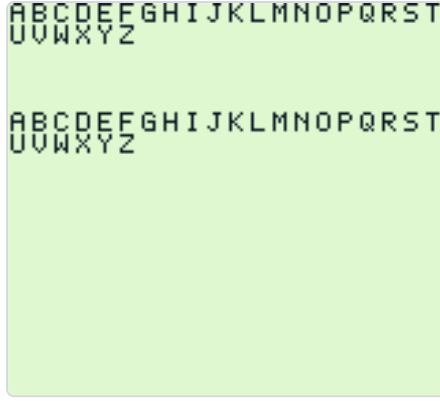
```
int main() {  
    // 1. Lignes 0 et 1 (le 0.8)  
    for (uint8_t i = 0; i < 20; i++) {    // A à T : ligne 0  
        poser(i, 0, ALPHABET[i]);  
    }  
    for (uint8_t i = 20; i < 26; i++) {    // U à Z : ligne 1  
        poser(i - 20, 1, ALPHABET[i]);  
    }  
    // 2. Lignes 5 et 6 (le 0.8.1)  
    for (uint8_t i = 0; i < 20; i++) {    // A à T : ligne 5  
        poser(i, 5, ALPHABET[i]);  
    }  
    for (uint8_t i = 20; i < 26; i++) {    // U à Z : ligne 6  
        poser(i - 20, 6, ALPHABET[i]);  
    }  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Tout l'alphabet deux fois : en haut, et trois lignes plus bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ lignes 0 et 1

✓ lignes 5 et 6

Ensuite : **0.9. Tout l'alphabet, avec une seule boucle** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++